



MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE
REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and
Microelectronics Department of Software and Automation Engineering

Lupasco Cristian FAF-233

Report

Laboratory work n.4

of Languages and Finite Automata

Checked by:

Cretu D., DISA, FCIM, UTM

Chişinău – 2024

Theory

A context-free grammar (CFG) describes a language with rules like 'A produces alfa' where A is a nonterminal and alfa is a sequence of symbols. In Chomsky Normal Form (CNF), every rule must be either:

- A produces two nonterminals

- A produces one terminal

- (Optionally) the start symbol produces the empty string

To convert any CFG into CNF, you perform these five steps in order:

- Remove all empty-string productions, redistributing nullable symbols into other rules.

- Eliminate any renaming rules where one nonterminal produces exactly one other nonterminal.

- Discard symbols and rules that can never be reached from the start symbol.

- Discard symbols and rules that cannot ultimately derive a string of terminals.

- For the remaining rules, introduce new nonterminals to stand for single terminals in longer right-hand sides, then break every right-hand side longer than two symbols into a chain of binary productions.

Conditions of the Problem

1. Learn about Chomsky Normal Form (CNF);
2. Get familiar with the approaches of normalizing a grammar;
3. Implement a method for normalizing an input grammar by the rules of CNF;
4. The implementation needs to be encapsulated in a method with an appropriate signature (also ideally in an appropriate class/type);
5. The implemented functionality needs executed and tested;
6. another BONUS point would be given if the student will make the aforementioned function to accept any grammar, not only the one from the student's variant;

Technical Implementation

```
def eliminate_epsilon(self):
    nullable = {A for A, prods in self.P.items() if []
                in prods}
    changed = True
    while changed:
        changed = False
        for A, prods in self.P.items():
            for prod in prods:
                if all(sym in nullable for sym in prod
                       ) and A not in nullable:
                    nullable.add(A)
                    changed = True
    newP: Dict[Symbol, List[Production]] = {}
    for A, prods in self.P.items():
        new_prods: Set[Tuple[Symbol, ...]] = set()
        for prod in prods:
            if prod == []:
                continue
            positions = [i for i, sym in enumerate(
                prod) if sym in nullable]
            for mask in range(1 << len(positions)):
                tmp = prod.copy()
                for i, pos in enumerate(positions):
                    if mask & (1 << i):
                        tmp[pos] = None
                filtered = [s for s in tmp if s is not
                           None]
                if filtered:
                    new_prods.add(tuple(filtered))
                new_prods.add(tuple(prod))
        newP[A] = [list(p) for p in new_prods]
    if self.S in nullable:
        newP.setdefault(self.S, []).append([])
    self.P = newP
```

Purpose: This method removes all empty-productions in two phases.

```
def eliminate_unit(self):
    # Build unit relation
    unit = {(A, A) for A in self.V}
    for A, prods in self.P.items():
        for prod in prods:
            if len(prod) == 1 and prod[0] in self.V:
                unit.add((A, prod[0]))
    changed = True
    while changed:
        changed = False
        for A, B in list(unit):
            for C, D in list(unit):
                if B == C and (A, D) not in unit:
                    unit.add((A, D))
                    changed = True
    newP: Dict[Symbol, List[Production]] = {A: [] for
        A in self.V}
    for A, B in unit:
        for prod in self.P.get(B, []):
            if not (len(prod) == 1 and prod[0] in self
                .V) and prod not in newP[A]:
                newP[A].append(prod)
    self.P = newP
```

Purpose: This function eliminates all unit productions (rules of the form $A \rightarrow B$). It first collects every pair (A, A) and every direct unit pair (A, B) . Then it computes the transitive closure: whenever (A, B) and (B, C) hold, add (A, C) . Finally, for each pair (A, B) in that closure, it copies every non-unit production $B \rightarrow$ into A 's productions, skipping any remaining unit rules and avoiding duplicates. The result is a grammar with no $A \rightarrow B$ rules but with equivalent terminal behavior.

```
def eliminate_inaccessible(self):
    reachable = {self.S}
    changed = True
    while changed:
        changed = False
        for A in list(reachable):
            for prod in self.P.get(A, []):
                for sym in prod:
                    if sym in self.V and sym not in
                        reachable:
                            reachable.add(sym)
                            changed = True
    self.V -= reachable
    self.P = {A: prods for A, prods in self.P.items()
               if A in reachable}
```

Purpose: This method removes any nonterminals and rules unreachable from the start symbol. It begins with the start in a “reachable” set, then repeatedly adds any nonterminal that appears on the right side of a production whose left side is already reachable. Once no new symbols can be added, it intersects the variable set with reachable and filters out all productions whose left side isn’t reachable. The result is a grammar stripped of any symbols and rules the start symbol can never derive.

```
def eliminate_nonproductive(self):
    productive = set()
    changed = True
    while changed:
        changed = False
        for A, prods in self.P.items():
            for prod in prods:
                if all(sym in self.T or sym in
                    productive for sym in prod):
                    if A not in productive:
                        productive.add(A)
                        changed = True
    good = self.V & productive
    self.V = good
    self.P = {
        A: [prod for prod in prods if all(sym in self.
            T or sym in good for sym in prod)]
        for A, prods in self.P.items() if A in good
    }
```

Purpose: This method prunes any nonterminals that cannot eventually derive a string of terminals. It repeatedly scans all productions, adding a variable to the “productive” set whenever it has a right side consisting solely of terminals or already-productive variables. Once no new productive symbols appear, it intersects the variable set with productive ones and filters out any productions involving nonproductive symbols. The result only keeps symbols that generate terminal strings.

```
def convert_to_cnf(self):
    newP: Dict[Symbol, List[Production]] = {}
    term_map: Dict[Symbol, Symbol] = {}
    counter = itertools.count(1)
    def get_term_var(t: Symbol) -> Symbol:
        if t not in term_map:
            var = f"T_{t}_{next(counter)}"
            term_map[t] = var
            self.V.add(var)
            newP[var] = [[t]]
        return term_map[t]
    for A, prods in self.P.items():
        newP.setdefault(A, [])
        for prod in prods:
            if len(prod) <= 1:
                newP[A].append(prod)
            else:
                transformed = [get_term_var(sym) if
                               sym in self.T else sym for sym in
                               prod]
                newP[A].append(transformed)
    cnfP: Dict[Symbol, List[Production]] = {}
    counter2 = itertools.count(1)
    for A, prods in newP.items():
        cnfP.setdefault(A, [])
        for prod in prods:
            if len(prod) <= 2:
                cnfP[A].append(prod)
            else:
                symbols = prod
                prev = symbols[0]
                for sym in symbols[1:-1]:
                    nxt = f"X_{next(counter2)}"
                    self.V.add(nxt)
                    cnfP.setdefault(nxt, []).append([
                        prev, sym])
                    prev = nxt
```



```
cnfP[A].append([prev, symbols[-1]])  
self.P = cnfP
```

Purpose: This method enforces the final CNF shape in two phases:

Terminal substitution Any right-hand side longer than one symbol is scanned; every terminal in it is replaced by a fresh nonterminal (e.g. Ta_1), and that new variable is given a single-terminal rule.

Binarization All remaining productions with length ≥ 2 are broken into binary chains: for a sequence $[X, Y, Z, \dots]$, it creates new variables $X_1 \rightarrow [X, Y]$, $X_2 \rightarrow [X_1, Z]$, etc., so that every rule ends up with exactly two nonterminals or a single terminal..

Visual Representation

```

=== Original Grammar ===
A -> b | aD | AS | bAB | ε
B -> a | bS
C -> AB
D -> BB
S -> aB | A | B

=== After eliminating empty productions ===
A -> S | bAB | A | bB | b | aD | AS
B -> a | b | bS
C -> AB | B
D -> BB
S -> A | B | aB | ε

=== After eliminating unit productions ===
A -> a | b | bS | aB | ε | bAB | bB | aD | AS
B -> a | b | bS
C -> AB | a | b | bS
D -> BB
S -> a | b | bS | aB | ε | bAB | bB | aD | AS

=== After eliminating inaccessible symbols ===
A -> a | b | bS | aB | ε | bAB | bB | aD | AS
B -> a | b | bS
D -> BB
S -> a | b | bS | aB | ε | bAB | bB | aD | AS

=== After eliminating non-productive symbols ===
A -> a | b | bS | aB | ε | bAB | bB | aD | AS
B -> a | b | bS
D -> BB
S -> a | b | bS | aB | ε | bAB | bB | aD | AS

=== Chomsky Normal Form ===
A -> a | b | T_b_1S | T_a_2B | ε | X_1B | T_b_1B | T_a_2D | AS
B -> a | b | T_b_1S
D -> BB
S -> a | b | T_b_1S | T_a_2B | ε | X_2B | T_b_1B | T_a_2D | AS
T a 2 -> a

```

Figure 1: Screenshot from PyCharm

Conclusions

By systematically applying the five transformation stages—eliminating empty-productions, removing unit rules, pruning inaccessible and non-productive symbols, then substituting terminals and binarizing—our Python implementation produces a grammar strictly in Chomsky Normal Form without changing its language. Each step preserves equivalence, and the final CNF grammar is ready for efficient parsing algorithms like CYK. This modular, list-based approach keeps multi-character symbols intact, avoids unwanted side effects, and cleanly separates concerns, making the code easy to extend or debug.